Temporal Machine Learning for In-Game MLB Pitcher Fatigue Detection

DATA 534: Applied Machine Learning, Final Presentation

Braeden Mefford

MS Data Science and Analytics
College of Charleston

April 20, 2026

Agenda

1. Motivation and problem
2. Objectives
3. Data: MLB Statcast
4. Data collection
5. Preprocessing and feature engineering
6. Label construction
7. Models and evaluation
8. Results
9. Case study: simulated trajectory
10. Uniqueness, limitations, future work

Motivation: Why Pitcher Fatigue?

The problem

- Pitcher fatigue drives performance decline and injury risk in MLB.
- Today, pitcher removal decisions rely on subjective cues: pitch count milestones, visual observation of mechanics, and the intuition of the pitching coach and catcher.
- Phillips (2017) showed that managers remove pitchers at round pitch counts with no statistical justification, indicating heuristic rather than data-driven decisions.
- Grantham et al. (2014) demonstrated that fatigue measurably alters pitching mechanics at the collegiate level, so biophysical signals exist in the data.

Why it matters

- MLB arms are multi-million dollar assets, and UCL injuries like Tommy John cost a full season or more.
- Teams already collect over four million pitch level records per season through Statcast.
- The data exists. A real-time, objective model does not.

Project Objectives

Primary goal

Build a pitch-by-pitch supervised classifier that flags pitcher fatigue in-game using only information available at the time of each pitch.

Specific objectives

1. Construct temporal features from raw Statcast pitches that quantify velocity decay, spin decay, release-point drift, and rising contact quality.
2. Define a forward-looking fatigue label that cannot leak into the feature set.
3. Compare three model architectures on a temporal train and test split. Logistic Regression, Random Forest, and XGBoost.
4. Identify the leading indicators of fatigue through feature importance analysis.
5. Deliver usable lead time so the model fires an alert before performance visibly collapses.

Dataset: MLB Statcast (Baseball Savant)

Source

- MLB Statcast is the radar and optical tracking system installed in every MLB ballpark.
- I accessed it through the `pybaseball` Python API, which pulls from Baseball Savant.
- I pulled seven regular seasons from 2019 through 2025.

Scale

- 4,603,293 pitches
- 93,942 pitcher-game appearances
- 2019 through 2023 for training, 2024 and 2025 held out for testing

Raw features captured for each pitch

- Velocity: `release_speed`, `effective_speed`
- Spin: `release_spin_rate`, `spin_axis`
- Release point: `release_pos_x/y/z`, `release_extension`
- Movement: `pfx_x`, `pfx_z`
- Plate location: `plate_x/z`, zone
- Batted ball: `launch_speed`, `launch_angle`, `bb_type`, `barrel`
- Contact quality: `xwOBA`, `xBA`

Data Collection

The download pipeline lives in `data_collection.py`.

- I queried `pybaseball.statcast()` in 30 day chunks per season. .
- I trimmed columns from roughly 90 down to about 38 relevant biomechanical and batted ball fields to keep memory usage manageable.
- Statcast occasionally returns duplicate rows at chunk boundaries, so I deduplicated on `(game_pk, pitcher, at_bat_number, pitch_number)`.
- Each season is cached as its own parquet file, then everything is combined into a single 4.6 million row dataset.

Challenges I encountered

- The API rate limits and times out often. I solved this with chunked requests and by enabling `pb.cache.enable()`.
- Schema drifts across years. Missing columns are silently skipped
- The 2020 season was shortened by COVID, so I built a per-year date window dictionary to handle it cleanly.
- Memory! The full dataset is around 3 GB, so I downcast float64 columns to float32 after feature engineering.

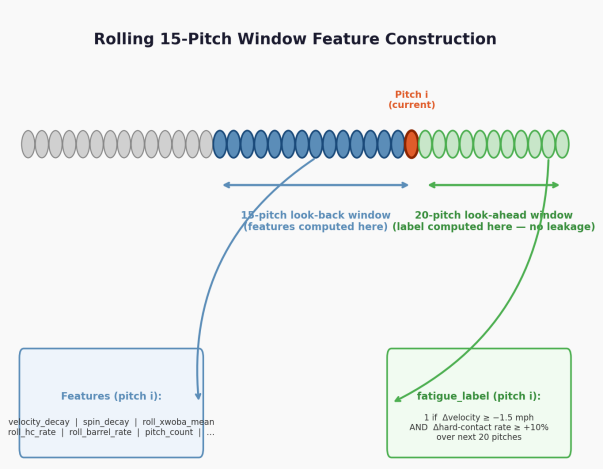
Preprocessing: Rolling Window Feature Construction

The core idea

- Pitches are ordered within each (`pitcher`, `game_pk`) group.
- For every pitch i, I build features from pitches $i - 14$ through i. This is a 15 pitch look-back window.
- Features for pitch i use only past and current information.

Per-game baselines

- The first 15 pitches of each appearance define per-pitcher, per-game baselines for velocity, spin, xwOBA, hard contact, and barrel rate.
- I also compute a fastball-only baseline separately so pitch mix does not confound the velocity signal.



The 15 pitch look-back builds features. A separate 20 pitch forward window defines the leak-free label described on the next slide.

Preprocessing: Over 30 Engineered Features in 6 Categories

Velocity, all pitches, fastball, offspeed

- `roll_velocity_mean / std / slope`
- `velocity_decay`, `velocity_drop_max`
- `fb_velocity_decay`, fastball only
- `os_velocity_decay`, offspeed only

Spin

- `roll_spin_mean / std / slope`
- `spin_decay`

Release point

- `roll_relx_std`, `roll_relz_std`
- `roll_ext_mean`, `roll_ext_std`

Movement

- `roll_pfxx / pfxz _mean / std`
- `roll_move_mean`

Location and control

- `roll_platex / platez _std`
- `roll_in_zone_rate`

Contact quality, new contribution versus prior work

- `roll_hc_rate`, exit velocity at or above 95 mph
- `hc_rate_rise`
- `roll_xwoba_mean`, `xwoba_rise`
- `roll_barrel_rate`, `roll_ld_rate`

Label Construction: Leak-Free Forward Window

A pitch is labelled fatigued if, in the next 20 pitches, both of the following conditions hold.

Velocity condition, fastball only

$$\overline{v}^{\text{FB}}_{i+1:i+20} \leq v^{\text{FB}}_{\text{baseline}} - 1.5 \text{ mph}$$

Only fastballs are compared against the fastball baseline. This prevents a shift toward more offspeed pitches from triggering a false fatigue flag.

Hard contact condition

$$\overline{\text{HC}}_{i+1:i+20} \geq \text{HC}^{\text{roll}}_{i} + 0.10$$

At least 10 percent more of the next 20 balls in play must have exit velocity of 95 mph or higher.

Key Features

- There is no leakage. Features use pitches 1 through i. The label uses pitches $i+1$ through $i+20$. These sets are independent.
- The two condition gate, velocity and contact, is stricter than prior work and filters out random pitch to pitch variation.
- Splitting by pitch type is a key advance over Kabra (2025), which mixed all pitch types and conflated fatigue with pitch selection.
- This change alone lifted ROC-AUC from 0.861 to 0.881 relative to the non-split baseline.

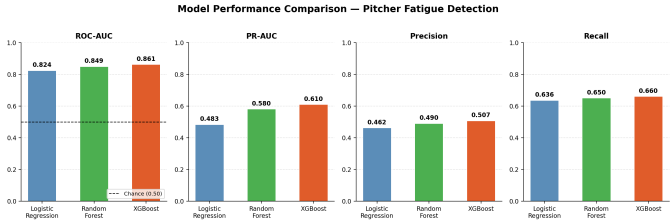
Methodology: Three Model Architectures

Model	Key hyperparameters	Rationale
Logistic Regression	StandardScaler with L2 penalty, C=0.1, `class_weight="balanced"`, `saga` solver, 1000 iterations	Linear baseline. The signed coefficients give an interpretable direction for each feature.
Random Forest	300 trees, `max_depth=10`, `min_samples_leaf=50`, balanced class weights	Captures non-linear interactions without feature scaling and gives a clean feature importance readout.
XGBoost	500 trees, `max_depth=6`, learning rate 0.05, subsample 0.8, `scale_pos_weight` set to n_-/n_+	Gradient boosted trees. Expected best performer on tabular temporal features of this kind.

Training and evaluation protocol

- Temporal split. 2019 through 2022 for training, 2023 as a validation set for threshold tuning, 2024 and 2025 for the final test.
- Metrics. ROC-AUC, PR-AUC as the primary metric for this imbalance, plus precision, recall, and F1.

Results: Model Comparison on the 2024 to 2025 Test Set



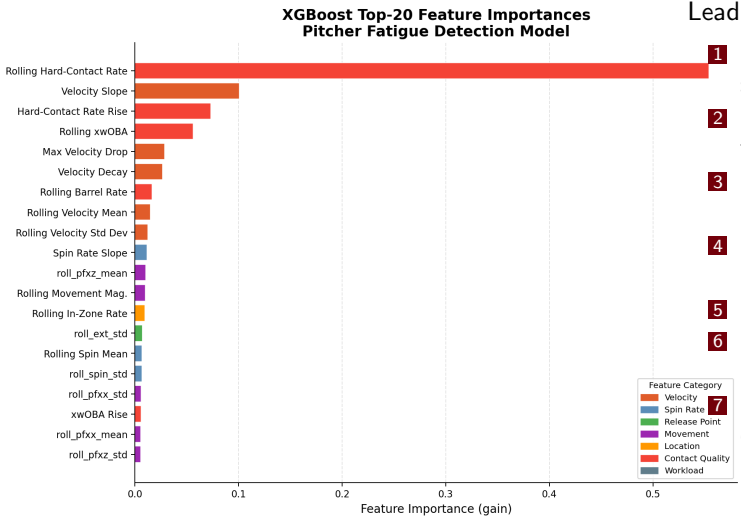
Model	ROC	PR	F1
LR	0.860	0.234	0.313
RF	0.873	0.251	0.326
XGB	0.881	0.271	0.335

XGBoost wins on every metric.

Best model, key numbers

- ROC-AUC of 0.881
- PR-AUC of 0.271, roughly nine times the 3 percent baseline rate
- Precision 31.5 percent, recall 35.7 percent
- F1 of 33.5 percent at a threshold of 0.850

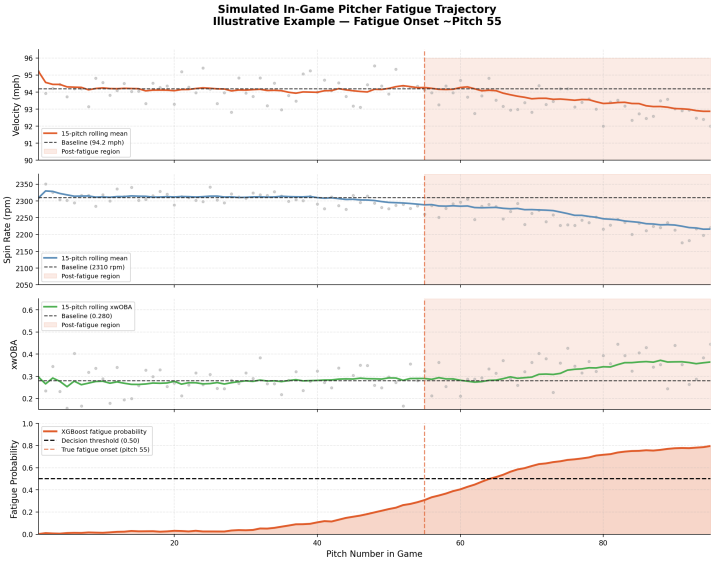
Results: XGBoost Top 20 Feature Importances



Leading indicators of fatigue

1. Rolling hard contact rate. Hitters are squaring it up.
2. Fastball velocity decay. The classic tell.
3. Pitch count. Cumulative workload matters.
4. Fastball velocity slope. Direction of recent change.
5. Hard contact rate at the raw level.
6. Spin velocity slope. This confirms Kabra (2025).
7. Rolling xwOBA mean. Expected damage is rising.

Case Study: Simulated In-Game Fatigue Trajectory



What is happening in this plot

- Velocity. The 15 pitch rolling mean drifts below the baseline.
- Spin rate. A slow decline across the outing.
- xwOBA. A sharp rise in the late innings, which means hitters are doing damage.
- Fatigue probability. Crosses the 0.5 threshold about 15 pitches after the true fatigue onset at pitch 55.

The key point is lead time. The model notes probable fatigue before major increase in opponent performance.

Advancement of Prior Work

Kabra (2025) in *Paripex, Indian Journal of Research*.

- Statcast combined with K-means clustering into 8 archetypes, then exponential smoothing, on a single 2021 season.
- Descriptive and unsupervised. No predictive model and no forward label.
- Main finding. Spin rate is the dominant signal. Velocity signal is weak at under half a standard deviation.

How this project advances that work

Dimension	Kabra (2025)	This work
Scale	1 season, about 700K pitches	7 seasons, 4.6M pitches
Features	Spin and velocity	Over 30 across 6 categories
Contact quality	None	xwOBA, barrel, line drive rate
Method	Unsupervised clustering	Supervised LR, RF, XGBoost
Label	Descriptive drift	Leak-free forward label
Output	Archetype summary	Per pitch probability with lead time

Limitations and Assumptions

Model limitations

- PR-AUC of 0.27 is a useful signal, but precision at the operating threshold is still modest.
- With a 3 percent positive rate, F1 optimisation is very sensitive to the threshold choice.
- Feature importance based on gain is correlational, not causal.

Assumptions

- I assume fatigue manifests within a 20 pitch horizon. Longer horizon fatigue cannot be captured by this label.
- I defined fatigue as a 1.5 mph fastball drop combined with a 10 percent rise in hard contact. These thresholds come from published work, not from the data.
- I assume the first 15 pitches represent a stable baseline. This fails for opener appearances under 15 pitches, which I filter out.
- There is no inter-game context. Days of rest, season workload, and prior-game pitch count are not yet in the feature set.

Conclusions and Future Work

Conclusions

- Rolling window temporal features successfully capture in-game pitcher fatigue dynamics directly from Statcast pitch data.
- Hard contact rate, fastball velocity decay, pitch count, and spin slope come through as the leading indicators. This is consistent with Kabra (2025) on spin, and it extends that work with contact quality signals.
- XGBoost reaches ROC-AUC of 0.881 on the held-out 2024 and 2025 test set and gives up to 20 pitches of lead time before measurable performance decline.

Future work

- Inter-game workload. Add days of rest, season to date pitch load, and a back to back appearance flag.
- Pitcher embedding. Per pitcher random effects or learned embeddings to personalize fatigue thresholds.
- Deployment. A real time dugout dashboard using the saved XGBoost model and a streaming Statcast style feed.

References

1. Phillips, D.C. (2017). Stopping on Nine: Evidence of Heuristic Managerial Decision-Making in MLB. *Southern Economic Journal*, 84(2).
2. Grantham, W.J. et al. (2014). The Impact of Fatigue on the Kinematics of Collegiate Baseball Pitchers. *Orthopaedic J. of Sports Medicine*, 2(6).
3. Whiteside, D. et al. (2016). Variations in Pitching Performance During a Major League Season. *Int. Conf. on Biomechanics in Sports*, vol. 33.
4. Woodham, M. et al. (2019). When to Pull Starting Pitchers in Major League Baseball: A Data Mining Approach. *Applied Data Science*.
5. Kabra, A. (2025). Evaluating Pitcher Fatigue Through Spin Rate Decline: A Statcast Data Analysis. *Paripex, Indian J. of Research*, 14(2).

Tools used. `pybaseball`, `pandas`, `scikit-learn`, `xgboost`, `matplotlib`, `seaborn`.
Code is available on GitHub. The QR code on the poster links to it.

Thank you.

Questions?

Braeden Mefford	bmefford19@gmail.com
Program	MS Data Science and Analytics, CofC
Course	DATA 534: Applied Machine Learning
Data	MLB Statcast 2019 to 2025, 4.6M pitches
Best model	XGBoost with ROC-AUC of 0.881